

Voice Agent: Brief

A Gemini-driven phone appointment scheduler on a real Twilio line

Working summary — concise, accurate

1 What it is

Voice Agent answers a real phone line with Twilio, uses Gemini 2.5 Flash (via Vertex AI) to figure out turn-by-turn what a caller wants (book, reschedule, or cancel an appointment) and extract the specific details, and writes the result to a real SQLite calendar with real conflict detection. It replaces a rigid button-press phone tree with natural spoken conversation for small businesses (clinics, salons, repair shops) that lose bookings to missed calls.

1.1 Verified vs. not verified

Verified for real: the app was run with the real Gemini engine and real SQLite, exposed over a public SSH tunnel, and driven through a genuine HMAC-SHA1-signed 3-turn booking conversation (opening call, natural-language booking request, phone number, confirmation). Gemini correctly resolved “tomorrow,” and a real appointment row was written to the database and confirmed by direct query, then cleaned up.

Not verified: an actual human voice call was never placed. Every test, including the tunnel run above, used hand-typed English text standing in for Twilio’s speech-to-text output, which is noisier in real use (mumbling, background noise, partial phrases).

2 Architecture

A caller’s speech flows: Twilio → `app.py` (FastAPI webhooks, signature verified with HMAC-SHA1) → `conversation.py` (the state machine) → Gemini (intent + slot extraction) and SQLite (calendar read/write) → back out as spoken TwiML. Per-call state (turn count, known slots, transcript) lives in a second SQLite database keyed by Twilio’s CallSid, since each `<Gather>` speech round-trip is a brand-new, stateless HTTP request.

3 Key components

- **app.py:** four endpoints (`/health`, `/voice/incoming`, `/voice/gather`, `/voice/status`). Verifies Twilio’s webhook signature (an HMAC-SHA1 hash keyed with the real auth token) when a public URL is configured. The production `app` object is built lazily so merely importing the module in tests never requires real credentials.
- **conversation.py:** the business-logic core. Fills slots (name, phone, date, time, duration) turn by turn, requires an explicit yes before writing anything, validates business hours and rejects

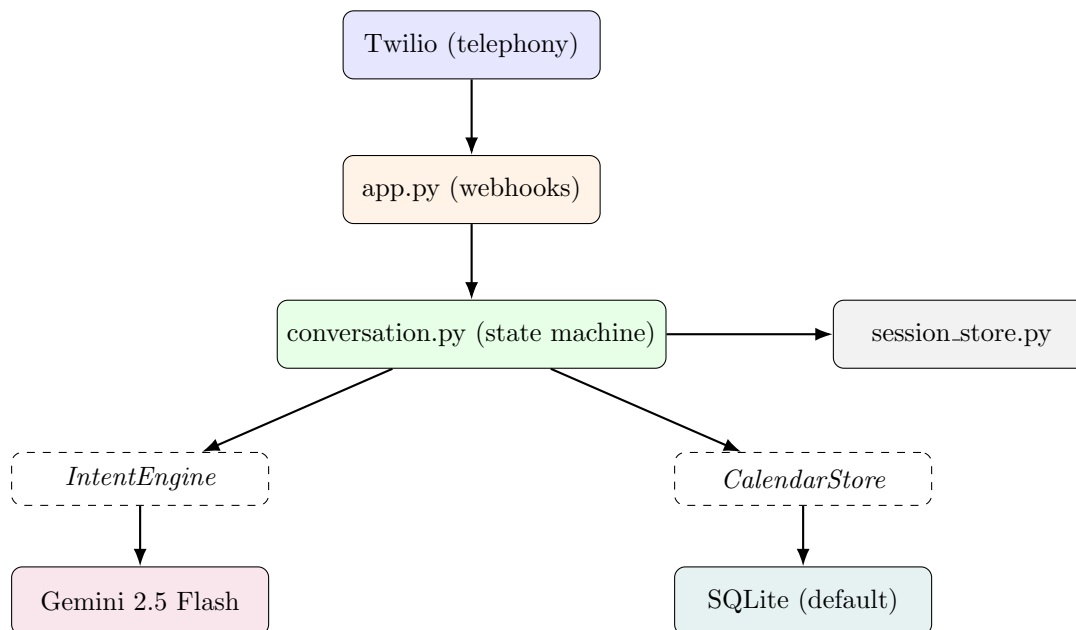


Figure 1: The conversation manager depends only on two interfaces (dashed), never directly on Gemini or SQLite — the one decision that makes cheap, deterministic testing possible (Section 4).

past times, handles booking conflicts by asking for another time, and enforces two safety caps: 3 consecutive unclear turns before handoff, 14 total turns before a forced handoff.

- **calendar_store.py:** `SQLiteCalendarStore` (default, the only backend actually exercised) uses a half-open interval overlap check ($a_start < b_end$ and $b_start < a_end$) so back-to-back appointments don't falsely conflict. `GoogleCalendarStore` is a second, fully implemented backend behind the same interface, selected automatically if Google credentials are configured — implemented but never run against a live calendar, since none were provided.
- **intent_engine.py:** `GeminiIntentEngine` calls Gemini 2.5 Flash with thinking disabled (a measured latency fix: default thinking added real seconds and 56 thinking tokens to a one-word answer; disabling it dropped the call to 1.8 seconds), a strict JSON response schema, and low temperature, all chosen because a live phone call cannot tolerate multi-second silences. `ScriptedIntentEngine` (in `eval/`) is a free, deterministic regex-based stand-in used only by tests and the evaluation harness, never in production.
- **session_store.py:** per-call state and full transcript, keyed by `CallSid`, so conversation survives across Twilio's stateless `<Gather>` round trips.
- **cli.py:** a human-operated tool that places a real, billed outbound test call; requires an explicit confirmation and a configured public URL, never runs automatically.

4 Testing strategy

Layer	Covers	Real API?
Calendar unit tests	Booking, conflicts, boundaries, reschedule, cancel	No
Endpoint tests (Test-Client)	Full HTTP flow, scripted engine standing in for Gemini	No
25 scripted scenarios	Real state machine + real temp SQLite, every action and edge case	No
<code>test_llm_intent.py</code>	The real Gemini engine, live credential	Yes (6 calls)
<code>test_twilio_integration.py</code>	Signature HMAC check + one read-only lookup	Partly

54 tests, 54 passed, 0 failed (47 free/fast by default, 7 real-API tests run separately by design). The interface-based design (`IntentEngine/CalendarStore`) is what makes the 25-scenario harness and most of the suite run without touching the billed Gemini API. To confirm the retry-cap and turn-cap scenarios were not passing vacuously, both invariants were deliberately broken in a throwaway run to verify the corresponding tests actually failed, then reverted.

Real API cost across the whole build and verification history: 12 Gemini calls (under a tenth of a cent, estimated from token rates) and three read-only Twilio REST calls (\$0.00, since Twilio does not bill read-only calls; no call or SMS was ever placed).

5 Notable engineering challenges

1. Twilio's `<Gather>` only re-requests its action URL on captured speech, not silence, by default — would have made the unclear-speech retry/handoff logic unreachable for a silent caller. Fixed with `action_on_empty_result="true"`.
2. Testing an LLM-driven system affordably: solved via the `IntentEngine` interface and a free scripted test double.
3. Relative-date parsing broke test reproducibility until a single injectable `now_fn` was threaded through the conversation manager and every scenario pinned to one fixed reference time.
4. An early scenario draft let the caller correct a bad time before confirming it, so the business-hours validation (which only runs after confirmation) never actually fired; caught by tracing turn-by-turn state and fixed by forcing an explicit confirming turn first.
5. Gemini's default thinking mode was too slow for a live phone turn; disabling it plus a strict JSON schema fixed both latency and parse reliability.

6 Honest gaps

- No timezone handling — one implicit local timezone assumed for everyone.
- The real Gemini engine has only been proven against clean, typed English, never noisy real speech-to-text output.

- The Google Calendar adapter is fully implemented but never run against a live calendar (no credentials provided).
- SQLite is single-writer; a multi-line deployment would need Postgres or WAL mode plus real concurrency testing.
- An actual human voice call was never placed; the closest proof is the HMAC-signed public-tunnel text conversation described above.

7 Glossary

Twilio

Cloud telephony platform: answers/places calls, converts speech to/from text.

Twiml

The XML dialect Twilio expects back, telling it what to say/do next.

Vertex AI

Google Cloud's platform for calling Gemini and other models.

HMAC-SHA1

A keyed cryptographic hash used to verify a webhook request is authentic and untampered.

Half-open interval

A range including its start but excluding its end; used so appointments that touch at the boundary don't falsely conflict.

Test double

A stand-in object used only in tests, never in production (here, `ScriptedIntentEngine`).

Dependency injection

Passing dependencies in from outside rather than constructing them internally, so tests can substitute fakes.